

AURA-8 System Architecture & Specification

Problem Statement 8: AI for Industrial Knowledge Intelligence

1. High-Level Architecture Overview

AURA-8 is built using a decoupled full-stack architecture combining a Next.js 15 App Router frontend and a Python 3.11 FastAPI backend connected directly to the Google Gemini API (gemini-flash-latest).

Subsystem	Technology	Core Responsibility
Client Layer	Next.js 15, React 19, Tailwind CSS	Gateway Landing Page, Persistent Chatbot, Canvas Knowledge Gra
Backend API Layer	Python 3.11, FastAPI, Uvicorn	PDF parsing, text chunking, TF-IDF vector indexing (:8000)
RAG & LLM Engine	Google Gemini API (gemini-flash-latest)	Grounded Q&A synthesis with page-level text citations
Parser Engine	PyPDF (pypdf), RegEx Entity Parser	Extracting tags (P-101A, V-301) and OISD/Factory Act directives

2. Ingestion & RAG Execution Flow

1. User uploads an industrial PDF/TXT file via the Next.js dropzone.
2. FastAPI receives file at POST /api/upload and invokes PyPDF text extraction.
3. Extracted text is split into 600-character chunks with 100-character overlap and indexed.
4. Entity Parser extracts equipment tags (e.g. P-101A) and regulatory codes to populate the 2D Knowledge Graph.
5. When the user asks a question, FastAPI calculates cosine similarity to fetch top 3 text chunks.
6. Context is sent to <https://generativelanguage.googleapis.com/v1beta/models/gemini-flash-latest:generateContent> for grounded answer synthesis.

3. Gateway & Zero-Data Mode Specifications

- **Start Clean Workspace:** Workspace starts with 0 data. Infographics, Knowledge Graph nodes, and RAG indexes populate ONLY after the first document upload.
- **Guest Demo Mode:** Invokes POST /api/mode/demo to load pre-configured sample hydrocracker data for instant evaluator review.